

Product & Architecture Requirements v1.0 Multi-Tenant Appointment Booking SaaS Platform

Document Status: Approved & Frozen Version: 1.0 Purpose: Product and Architecture

Baseline Audience: Founders, Product Managers, Architects, Engineering Leads, Developers, QA Engineers, DevOps Engineers

1. Executive Summary

This document defines the approved product vision, business rules, architecture decisions, tenant model, security model, booking model, ownership model, scalability strategy, and development roadmap for the Appointment Booking SaaS Platform.

This document is the official source of truth before implementation begins.

Its purpose is to ensure all stakeholders share a common understanding of:

What is being built Why it is being built How tenants operate How security operates How appointments operate How ownership works How the platform scales What belongs in MVP What is intentionally excluded from MVP

This document intentionally avoids implementation details such as:

APIs DTOs Controllers Repositories Database schemas ORM models Source code Infrastructure automation

Those belong in future technical design documents.

1. Product Vision Vision Statement

Build a modern multi-tenant appointment booking SaaS platform that allows independent service businesses to manage appointments, staff, availability, clients, and branding from a single shared platform.

The platform must provide a professional booking experience while maintaining strong tenant isolation and low operational cost.

Core Product Characteristics

The platform uses:

One mobile application One backend One shared PostgreSQL database Multiple independent businesses Strong tenant isolation Tenant-specific branding Client membership in multiple businesses Secure tenant switching

No separate applications are created per business.

No separate backend deployments are created per business.

No separate databases are created per business.

Supported Business Types

The platform is designed for appointment-based businesses including:

Barbers Hair salons Nail artists Beauty providers Clinics Therapists Coaches Personal trainers Consultants Future appointment-based businesses

The architecture should remain generic enough to support additional service businesses without

redesign.

1. Business Goals Client Value

Clients should be able to:

Discover services offered by businesses they belong to View available appointment times Book appointments Manage appointments Receive notifications Belong to multiple businesses Switch businesses easily Business Owner Value

Business owners should be able to:

Manage staff Manage services Manage schedules Manage appointments Manage branding Manage clients Grow their business without managing infrastructure Staff Value

Staff should be able to:

View schedules Manage availability Manage assigned appointments Receive appointment updates Platform Operator Value

The platform operator should be able to:

Operate a single SaaS platform Support multiple businesses Maintain strong tenant isolation Scale infrastructure efficiently Avoid custom deployments per business

The platform prioritizes:

1. Simplicity

Prefer straightforward solutions over complex solutions.

1. Security

Backend validation must be the source of truth.

1. Tenant Isolation

Business data must remain isolated.

1. Maintainability

The architecture should be understandable and easy to evolve.

1. Development Speed

The platform should be practical to build and iterate on.

1. Low Operational Cost

The MVP should be inexpensive to operate.

1. Future Extensibility

The architecture should allow future growth without redesign.

1. MVP Scale Target

The MVP is expected to support:

Up to 10 businesses Up to 1,000 users Up to 50,000 appointments MVP Infrastructure

The MVP should comfortably operate using:

One backend instance One PostgreSQL database One mobile application Operational Cost Goal

Preferred operational cost:

Under \$50/month

Excluding:

Apple Developer Program Google Play Developer Account 6. Future Growth Target

The platform should support future growth to:

100+ businesses 10,000+ users Hundreds of thousands of appointments

without redesigning:

Tenant model Membership model Security model Booking model Future Scaling Strategy

Scaling should occur through:

Additional backend instances Improved indexing Background workers Selective caching Infrastructure upgrades

The platform should not require a complete architectural rewrite.

1. Multi-Tenant Architecture Tenant Definition

A tenant represents an independent business.

Examples:

Ahmad Barber Yousef Barber Luxury Nails Prime Therapy Fit Coach Studio

Each tenant operates independently.

Approved Multi-Tenant Model

The platform uses:

Shared Database

One PostgreSQL database.

Shared Schema

One database schema.

Tenant Ownership

Tenant-owned records are logically isolated.

Why This Model Was Selected

Advantages:

Simpler operations Lower cost Easier onboarding Easier reporting Easier maintenance Faster development Rejected Alternative: Database Per Tenant

Rejected because:

Higher cost Operational complexity Migration complexity Monitoring complexity Rejected
Alternative: Backend Per Tenant

Rejected because:

Higher maintenance burden More deployments SaaS inefficiency 8. Tenant Ownership
Principle Principle

Every business entity belongs to exactly one tenant.

The tenant is the ownership boundary of the platform.

Examples

The following entities belong to one tenant:

Services Service categories Staff profiles Client profiles Appointments Availability
rules Schedules Time off records Invitations Notifications Branding settings Audit
records Resource Sharing

Resources are not shared between tenants in the MVP.

A service belonging to one tenant cannot automatically belong to another tenant.

A staff profile belonging to one tenant does not automatically belong to another tenant.

Benefits

This principle simplifies:

Security Authorization Auditing Reporting Compliance Future maintenance 9. User Model Global
User Identity

A user represents a person.

A user has one global identity.

Why This Approach Was Selected

Rejected approach:

Separate user systems for:

Owners Staff Clients

Problems:

Duplicate accounts Poor user experience Complex onboarding Difficult tenant switching Approved
Approach

One user can simultaneously be:

Owner in Tenant A Staff in Tenant B Client in Tenant C

Roles are tenant-specific.

1. Membership Model Membership Definition

A membership connects:

User Tenant Role

A membership defines access.

Membership Lifecycle

Typical lifecycle:

Invitation created Invitation accepted Membership created Membership active Membership modified Membership removed Source of Truth

Memberships are the source of truth for access control.

No user may access a tenant without membership.

1. User Roles Platform Admin

Platform-level operator.

Responsibilities:

Tenant support Platform operations Abuse management

Restrictions:

Must be heavily audited Owner

Business owner.

Responsibilities:

Business management Staff management Service management Branding management Admin

Business manager.

Responsibilities:

Daily operations Appointment management Staff support Staff

Service provider.

Responsibilities:

Availability management Assigned appointments Client

Customer.

Responsibilities:

Booking appointments Managing own appointments